数据库设计与开发
Database Design & Development

计算机学院　单纯
sherryshan@bit.edu.cn
2024年3月

数据库的完整性
Database Integrity

计算机学院　单纯
sherryshan@bit.edu.cn
2024年5月

内容概览

- 9.1 数据库的完整性概述
- 9.2 实体完整性
- 9.3 参照完整性
- 9.4 用户定义的完整性
- 9.5 触发器
- 9.6 小结

9.1 数据库的完整性概述（1）

- 数据库的完整性是指数据的正确性、有效性和相容性，其主要目的是防止错误的数据进入数据库
  - 所谓正确性是指数据的合法性
  - 所谓有效性是指数据是否属于所定义的有效范围
  - 相容性是指表示同一事实的两个数据应相同，不一致就是不相容

9.1 数据库的完整性概述（2）

- 9.1.1 完整性约束条件
- 9.1.2 实现数据完整性的方法

9.1.1 完整性约束条件（1）

- 在关系数据库中，对数据的各种限制是以完整性约束条件的形式在关系数据库模式中指定的
  - 加在数据上的语义约束条件称为数据库完整性约束条件
  - 它一般是对数据库中数据本身的某种语义限制、数据间的逻辑约束和数据变化时所遵循的规则等

9.1.1 完整性约束条件（2）

一、列级约束
二、元组约束
三、关系约束

一、列级约束

- 主要是对属性的数据类型、数据格式和取值范围、精度等的约束。具体包括
  - 对数据类型的约束，包括数据类型、长度、精度等的约束。例如学生姓名的数据类型是字符型，长度是20
  - 对数据格式的约束，例如规定日期的格式为YYYY/MM/DD
  - 对取值域的约束，例如学生成绩的取值范围必须是0～100
  - 对空值的约束

二、元组约束

- 一个元组是由若干个属性组成的，元组级约束就是元组中各个属性之间的约束关系。例如订货关系中发货日期不能小于订货日期，发货量不得超过订货量等

三、关系约束

- 关系约束是指一个关系的各个元组之间、或者多个关系之间存在的各种联系或约束。常见的关系约束有
  - 实体完整性约束
  - 参照完整性约束
  - 函数依赖约束
  - 统计约束

9.1.2 实现数据完整性的方法（1）

- 数据库的完整性控制是DBMS的基本功能之一，DBMS中实现完整性控制机制的子系统称为完整性子系统
- 为维护数据库的完整性，DBMS在完整性约束控制方面应当具有
  - 一、提供定义完整性约束条件的机制
  - 二、提供完整性检查的方法
  - 三、违约处理

一、提供定义完整性约束条件的机制

- 主要提供完整性约束条件的定义功能，例如由SQL的DDL语言来实现

二、提供完整性检查的方法

- DBMS检查数据是否满足完整性条件的机制称为完整性检查。一般在数据更新语句（插入、删除和修改）执行后开始检查，也可以在事务提交时检查，检查这些操作执行后数据库中的数据是否违反了完整性约束条件

三、违约处理

- 违约处理功能主要是对违反完整性约束条件的业务操作采取相应措施，以保证数据的完整性。违约处理操作包括拒绝执行该操作或级联执行等
- 对于违反实体完整性约束和用户定义完整性约束的操作，数据库管理系统一般都采用拒绝执行的方式进行处理
- 对于违反参照完整性的操作，系统一般会执行一些附加操作，以保证数据库的状态是正确的

9.1.2 实现数据完整性的方法（2）

- 在关系数据库系统中，数据完整性控制策略包括规则、默认值、约束、触发器和存储过程等
  - （1）默认值
    - 如果在插入行中没有指定列的值，那么默认值指定列中所使用的值，例如：自动增长值，内置函数、数学表达式等

9.1.2 实现数据完整性的方法（3）

- （2） 约束
  - 约束是自动强制数据完整性的方法
  - 约束定义关系列中允许值的规则，是通用的强制完整性的标准机制
  - 使用约束优于使用触发器、规则和默认值
- （3）规则
  - 规则是大多数数据库系统中一个向后兼容的功能，用于执行一些与CHECK约束相同的功能。规则以单独的对象创建，然后绑定到列上
- （4）触发器
  - 触发器是数据库系统中强制业务规则和数据完整性的主要机制

9.1 数据库的完整性概述（3）

- 声明式数据完整性
  - 作为对象定义的一部分来定义数据必须达到的标准
  - DBMS自动强制完整性
  - 通过使用约束、默认和规则来实现

9.1 数据库的完整性概述（4）

- 过程式数据完整性
  - 在脚本中定义数据必须达到的标准
  - 在脚本中强制完整性
  - 通过使用触发器和存储过程来实现
  - 可在客户端或服务器用其他编程语言和工具来实现

9.2 实体完整性（1）

- 实体完整性规则规定
  - 主键的值不能取空值
  - 主键的值唯一
- 实现方法
  - 通过对主键值的约束实现实体完整性

9.2 实体完整性（2）

- 9.2.1 实体完整性的定义
- 9.2.2 实体完整性检查和违约处理

9.2.1 实体完整性的定义（1）

- 关系模型的实体完整性用PRIMARY KEY定义
  - CREATE TABLE中用PRIMARY KEY定义
    - 单属性构成的主键有两种说明方法
      - 定义为列级约束条件、定义为表级约束条件
    - 对多个属性构成的主键只有一种说明方法
      - 定义为表级约束条件
- PRIMARY KEY约束可以作为表定义的一部分在定义表时定义，也可以在表创建之后再添加

9.2.1 实体完整性的定义（2）

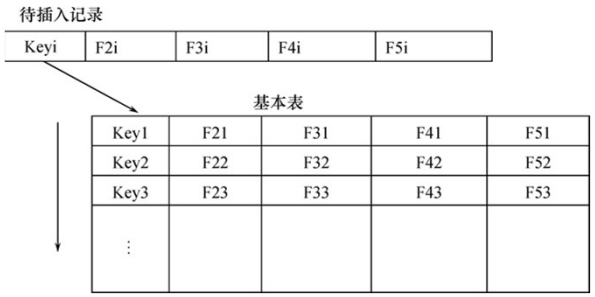
- 如果数据表已有PRIMARY KEY约束，还可以对其进行修改或删除
- 为了实施实体完整性，一般地，系统将在主键属性上自动创建唯一的索引来强制唯一性约束

9.2.2 实体完整性检查和违约处理（1）

- 定义表的主键后，对该表插入记录或者对主键进行更新操作时，DBMS自动进行实体完整性检查。包括
  - 检查主键是否唯一
    - 如果不唯一则拒绝进行插入或修改
  - 检查主键各个属性（字段）值是否为空
    - 如果有空的字段值，则拒绝操作

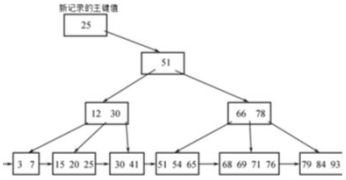
9.2.2 实体完整性检查和违约处理（2）

- 检查主键值是否唯一的一种方法是进行全表扫描



9.2.2 实体完整性检查和违约处理（3）

- 全表扫描十分耗时。为了避免对基本表进行全表扫描，RDBMS核心一般都在主键上自动建立一个索引，通过索引查找基本表中是否已经存在新的主键值，提高效率
  - B+树索引



9.3 参照完整性（1）

- 参照完整性给出了数据表之间互相关联的基本要求，其核心是不允许引用不存在的元组，也就是说，外键字段的取值要么为空值，要么就是被引用关系中元组的对应值

9.3 参照完整性（2）

- 9.3.1 参照完整性定义
- 9.3.2 参照完整性检查和违约处理

9.3.1 参照完整性定义

- 参照完整性在CREATE TABLE中用FOREIGN KEY和REFERENCES定义
  - FOREIGN KEY短语指明表中的哪些字段是外键，REFERENCES短语指明该表中的外键所参照的数据表名和字段名
  - FOREIGN KEY约束可以作为表定义的一部分在创建表时创建，也可以在已有表中创建FOREIGN KEY约束

9.3.2 参照完整性检查和违约处理（1）

- 可能破坏参照完整性的情况及违约处理

被参照表（例如Student）	参照表（例如SC）	违约处理
可能破坏参照完整性 ←	插入元组	拒绝
可能破坏参照完整性 ←	修改外键值	拒绝
删除元组 →	可能破坏参照完整性	拒绝/级联删除/设置为空值
修改主键值 →	可能破坏参照完整性	拒绝/级联修改/设置为空值

9.3.2 参照完整性检查和违约处理（2）

一、参照关系中外键空值的问题
二、在参照关系中插入元组的问题
三、在参照关系中修改元组的问题
四、在被参照关系中删除元组的问题
五、在被参照关系中修改主键值的问题

一、参照关系中外键空值的问题

- 需要定义外键是否允许为空值
  - 如果外键是其主键的组成部分，外键值不允许为空
  - 否则可以根据具体的语义确定外键值是否允许空值

二、在参照关系中插入元组的问题

- 受限插入
  - 向参照关系中插入新的元组，但该元组的属性值在被参照关系中不存在，则系统拒绝插入该元组
  - 只有被参照关系中存在相应的元组，系统才能执行插入操作
- 递归插入
  - 首先向被参照关系插入相应的元组，其主键值等于参照关系插入元组的外键值
  - 然后再向参照关系插入该元组

三、在参照关系中修改元组的问题

- 修改操作可以先删除、后插入
- 按照“在参照关系中插入元组的问题”进行处理

四、在被参照关系中删除元组的问题（1）

- 级联删除（CASCADES）
  - 将参照关系中所有外键值与被参照关系中要删除元组之间值相同的元组一起删除
  - 如果参照关系同时又是另一个关系的被参照关系，这种删除操作会继续级联下去
- 受限删除（RESTRICTED）
  - 仅当参照关系中没有任何元组的外键值与被参照关系中要删除元组的主键值相同时，系统才执行删除操作，否则拒绝删除操作

四、在被参照关系中删除元组的问题（2）

- 置空值删除
  - 删除被参照关系的元组，并将参照关系中相应元组的外键值置为空值
- 这三种方法都保证了参照完整性约束，但具体哪种方法正确，取决于应用环境的语义，需要根据实际应用的业务规则决定具体的违约处理措施

五、在被参照关系中修改主键值的问题

- 当修改被参照关系的主键时，除了需要保证主键的唯一性和非空性，还需要检查参照关系，是否存在这样的元组，其外键值等于被参照关系要修改的主键值

9.3.2 参照完整性检查和违约处理（3）

- 数据库管理系统在实现参照完整性时，除了要提供定义主键、外键的机制之外，还需要提供不同的违约处理策略供用户选择
- 一般地，当对参照表和被参照表的操作违反了参照完整性，系统选择默认策略，即拒绝执行
- 如果想让系统采用其他的策略则必须在创建表时显示地加以说明

9.4 用户定义的完整性（1）

- 用户定义的完整性就是针对某一具体应用的数据必须满足的语义要求
- RDBMS提供，而不必由应用程序承担

9.4 用户定义的完整性（2）

- 9.4.1 属性上的约束条件的定义
- 9.4.2 属性上的约束条件检查和违约处理
- 9.4.3 元组上的约束条件的定义
- 9.4.4 元组上的约束条件检查和违约处理

9.4.1 属性上的约束条件的定义

- CREATE TABLE时定义
  - 列值非空（NOT NULL）
  - 列值唯一（UNIQUE）
  - 检查列值是否满足一个布尔表达式（CHECK）

9.4.2 属性上的约束条件检查和违约处理

- 插入元组或修改属性的值时，RDBMS检查属性上的约束条件是否被满足
  - 如果不满足则操作被拒绝执行

9.4.3 元组上的约束条件的定义

- 在CREATE TABLE时可以用CHECK短语定义元组上的约束条件，即元组级的限制
- 同属性值限制相比，元组级的限制可以设置不同属性之间的取值的相互约束条件

9.4.4 元组上的约束条件检查和违约处理

- 插入元组或修改属性的值时，RDBMS检查元组上的约束条件是否被满足
  - 如果不满足则操作被拒绝执行

9.4 用户定义的完整性（3）

- 完整性约束命名子句
  - CONSTRAINT 约束

    CONSTRAINT <完整性约束条件名>
    [PRIMARY KEY短语
    |FOREIGN KEY短语
    |CHECK短语]

例题（1）

- 建立学生登记表Student，要求学号在90000~99999之间，姓名不能取空值，年龄小于30，性别只能是“男”或“女”

例题（2）

```
CREATE TABLE Student
（Sno  NUMERIC（6）
  CONSTRAINT C1 CHECK （Sno BETWEEN 90000 AND 99999），
  Sname  CHAR（20）
  CONSTRAINT C2 NOT NULL，
  Sage  NUMERIC（3）
  CONSTRAINT C3 CHECK （Sage < 30），
   Ssex  CHAR（2）
   CONSTRAINT C4 CHECK （Ssex IN （‘男’，‘女’）），
   CONSTRAINT StudentKey PRIMARY KEY（Sno）
 ）；
```

例题（3）

- 在Student表上建立了5个约束条件，包括主键约束（命名为StudentKey）以及C1、C2、C3、C4四个列级约束

9.4 用户定义的完整性（4）

- 完整性约束命名子句（续）
  - 使用ALTER TABLE语句修改表中的完整性限制

例题（1）

- 修改表Student中的约束条件，要求学号改为在900000~999999之间，年龄由小于30改为小于40
  - 可以先删除原来的约束条件，再增加新的约束条件

例题（2）

```
ALTER TABLE Student
    DROP CONSTRAINT C1；
ALTER TABLE Student
    ADD CONSTRAINT C1 CHECK （Sno BETWEEN 900000 AND 999999）；

ALTER TABLE Student
    DROP CONSTRAINT C3；
ALTER TABLE Student
    ADD CONSTRAINT C3 CHECK （Sage < 40）；
```

9.5 触发器（1）

- 数据库系统一般提供两种主要机制来实现业务规则和数据完整性
  - 约束
    - 完整性约束机制在检测出违反约束条件的操作后，只能作简单的动作。例如，拒绝操作
  - 触发器
    - 触发器是用户定义在关系数据表上的一类由事件驱动的特殊过程，用编程的方法实现复杂的业务规则
    - 触发器比约束更加灵活，可以实现一般的数据完整性约束实现不了的复杂的完整性约束，具有更精细和更强大的数据控制能力
    - 触发器常常用于强制业务规则和数据完整性

9.5 触发器（2）

- 9.5.1 触发器概述
- 9.5.2 定义触发器
- 9.5.3 使用触发器
- 9.5.4 删除触发器

9.5.1 触发器概述

- 触发器是一种特殊类型的存储过程，在对表或视图发出 UPDATE、INSERT 或 DELETE 语句时自动执行
- 可用触发器完成很多数据库完整性保护的功能
  - 实现复杂的业务规则
  - 实现比CHECK 约束更复杂的数据完整性
  - 比较数据修改前后的状态
  - 维护非规范化数据

9.5.2 定义触发器（1）

- SQL使用CREATE TRIGGER命令创建触发器，其一般格式为

```
CREATE TRIGGER <触发器名>
{ BEFORE | AFTER} <触发事件> ON <表名>
FOR EACH {ROW | STATEMENT}
[WHEN <触发条件>]
<触发动作体>
```

9.5.2 定义触发器（2）

- 定义触发器的语法说明
  1. 创建者：表的拥有者
  2. 触发器名，表名：触发器的目标表
  3. 触发事件：INSERT、DELETE、UPDATE

9.5.2 定义触发器（3）

- 定义触发器的语法说明（续）
  4. 触发时间
     - BEFORE
       - 表示在触发事件进行以前，判断触发条件是否满足
         - 若满足条件则先执行触发动作部分的操作
         - 然后再执行触发事件的操作
     - AFTER
       - 表示在触发事件完成之后，判断触发条件是否满足
         - 若满足条件则执行触发动作部分的操作
         - 如果触发事件因错误（如违反约束或语法错误）而失败，触发器将不会执行

9.5.2 定义触发器（4）

- 定义触发器的语法说明（续）
  5. 触发器类型
     - 行级触发器（FOR EACH ROW）
       - 对每一个修改的元组都会触发触发器的检查和执行
     - 语句级触发器（FOR EACH STATEMENT）
       - 只在SQL语句执行时候进行触发条件的检查和触发器的执行

9.5.2 定义触发器（5）

- 定义触发器的语法说明（续）
  6. 触发条件
     - 触发条件为真，省略WHEN触发条件
  7. 触发动作体
     - 触发动作体是满足触发器条件后，执行的一系列数据库操作
     - 如果触发动作体执行失败，激活触发器的事件就会终止执行，触发器的目标表或触发器可能影响的其他对象不发生任何变化

例题（1）

- 定义一个BEFORE行级触发器，为教师表Teacher定义完整性规则“教授的工资不得低于4000元，如果低于4000元，自动改为4000元”

例题（2）

```
CREATE TRIGGER Insert_Or_Update_Sal
    BEFORE INSERT OR UPDATE ON Teacher
    /*触发事件是插入或更新操作*/
    FOR EACH ROW                /*行级触发器*/
    AS BEGIN      /*定义触发动作体，是PL/SQL过程块*/
        IF （new.Job='教授'）AND （new.Sal < 4000）
        THEN
            new.Sal :=4000;
        END IF；
    END；
```

例题（3）

- 定义AFTER行级触发器，当教师表Teacher的工资发生变化后就自动在工资变化表Sal_log中增加一条相应记录

例题（4）

- 首先建立工资变化表Sal_log

```
CREATE TABLE Sal_log
    （Eno   NUMERIC（4）  REFERENCES
Teacher（Eno），
      Sal    NUMERIC（7，2），
      Username  CHAR（10），
      Date   TIMESTAMP
    ）；
```

例题（5）

```
CREATE TRIGGER Insert_Sal
    AFTER INSERT ON Teacher
        /*触发事件是INSERT*/
    FOR EACH ROW
    AS BEGIN
       INSERT INTO Sal_log VALUES（
       new.Eno，new.Sal，CURRENT_USER，
       CURRENT_TIMESTAMP）；
    END；
```

例题（6）

```
CREATE TRIGGER Update_Sal
    AFTER UPDATE ON Teacher
      /*触发事件是UPDATE */
    FOR EACH ROW
    AS BEGIN
       IF （new.Sal <> old.Sal）
          THEN INSERT INTO Sal_log VALUES（
           new.Eno，new.Sal，CURRENT_USER，
           CURRENT_TIMESTAMP）；
       END IF；
    END；
```

9.5.3 使用触发器

- 触发器的执行，是由触发事件激活的，并由数据库服务器自动执行
- 一个数据表上可能定义了多个触发器
  - 同一个表上的多个触发器激活时遵循如下的执行顺序
    - （1） 执行该表上的BEFORE触发器
    - （2） 激活触发器的SQL语句
    - （3） 执行该表上的AFTER触发器

例题

- 执行修改某个教师工资的SQL语句，激活上述定义的触发器

  UPDATE Teacher SET Sal=800 WHERE Ename=‘陈平’；
  - 执行顺序是
    - 执行触发器Insert_Or_Update_Sal
    - 执行SQL语句“UPDATE Teacher SET Sal=800 WHERE Ename=‘陈平’；”
    - 执行触发器Insert_Sal
    - 执行触发器Update_Sal

9.5.4 删除触发器

- 删除触发器的SQL语法

  DROP TRIGGER <触发器名> ON <表名>；
  - 触发器必须是一个已经创建的触发器，并且只能由具有相应权限的用户删除

例题

- 删除教师表Teacher上的触发器Insert_Sal

  DROP TRIGGER Insert_Sal ON Teacher；

9.6 小结（1）

- 数据库的完整性是为了保证数据库中存储的数据是正确的
- RDBMS完整性实现的机制
  - 完整性约束定义机制
  - 完整性检查机制
  - 违背完整性约束条件时RDBMS应采取的动作

9.6 小结（2）

- 数据库的完整性
  - 实体完整性
  - 参照完整性
  - 用户定义的完整性

9.6 小结（3）

- 数据完整性控制策略
  - 规则
  - 默认值
  - 约束
  - 触发器
  - 存储过程

9.6 小结（4）

- 应综合考虑功能性和性能开销，来决定使用何种强制数据完整性的方法
  - 对于基本的完整性逻辑，例如有效值和维护表间的关系，最好使用声明式完整性约束。
  - 如果要维护复杂的、大量的、非主键或外键关系一部分的数据，必须使用触发器或存储过程（过程式数据完整性）

参考资料

- 第7、12章，周志逵 郭贵锁等编著，数据库系统原理，北京：清华大学出版社，2008
- 第五章，王珊 萨师煊，数据库系统概论（第5版）， 北京：高等教育出版社，2014

Thank You